

PRODUCTION IMPLEMENTATION GUIDE

WhatsApp Cloud API Integration Handbook

A complete, reusable blueprint for connecting a web application to the official Meta WhatsApp Business Platform—with secure webhooks, outbound messaging, templates, media, delivery tracking, opt-out handling, multi-tenant isolation, and optional AI automation.

Edition

September 2026

Reference stack

Node.js · TypeScript · Express · PostgreSQL

Audience

Engineering, security, product, and operations teams

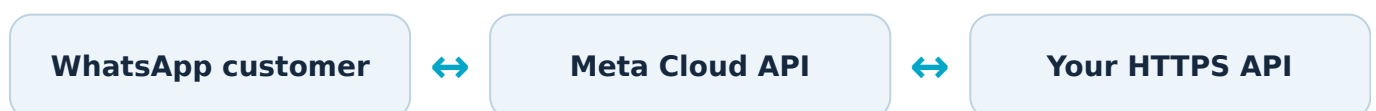
Purpose. This handbook is a vendor-neutral implementation reference derived from a production integration. It contains no credentials, customer data, or project-specific secrets. Adapt the data model and deployment steps to your own application.

Contents

1. Scope and architecture	11. Delivery receipts
2. Meta prerequisites	12. Consent and opt-out
3. Configuration and secrets	13. Multi-tenant isolation
4. Recommended data model	14. Optional AI agent
5. Secure webhook implementation	15. Reliability and security
6. WABA app subscription	16. Testing
7. Inbound message processing	17. Deployment
8. Outbound messages	18. Troubleshooting
9. 24-hour window and templates	19. Production checklists
10. Media and documents	20. Official references

1. Scope and architecture

This guide uses Meta's official **WhatsApp Cloud API**. It does not require Twilio or an unofficial browser automation library. The application receives inbound events through HTTPS webhooks and sends outbound messages through the Graph API.



A production implementation normally includes five layers:

1. **Channel configuration:** WABA, phone-number ID, access token, app secret, templates, and health state.
2. **Webhook gateway:** challenge verification, raw-body signature validation, parsing, and immediate acknowledgement.

3. **Message service:** send text, interactive messages, templates, and media through one provider abstraction.
4. **Persistence and workers:** transcript storage, idempotency ledger, durable outbox, retries, and delivery receipts.
5. **Business workflow:** customer support, sales intake, notifications, CRM updates, and optional AI responses.

2. Meta prerequisites

1. Create or use a verified Meta Business Portfolio.
2. Create a Meta Developer app and add the **WhatsApp** product/use case.
3. Create or connect a WhatsApp Business Account (WABA).
4. Add and verify the business phone number.
5. Record the **App ID**, **WABA ID**, and **Phone Number ID**.
6. Create a system user and generate a production access token with the minimum required WhatsApp permissions.
7. Expose a stable public HTTPS webhook endpoint.
8. Configure the webhook callback URL and a private verify token in the Meta App Dashboard.
9. Subscribe to the `messages` webhook field.
10. Explicitly subscribe the app to each WABA using `/ {WABA_ID} / subscribed_apps`.

Common production gap: configuring the callback URL is not enough. If the app is not attached to the WABA through `subscribed_apps`, inbound messages may never reach your webhook.

3. Configuration and secrets

Store credentials in a managed secret store. Never commit them, return them from settings APIs, place them in client-side variables, or print them in logs.

Variable	Purpose	Sensitivity
WHATSAPP_ACCESS_TOKEN	Bearer token used for Graph API calls	Secret
WHATSAPP_APP_SECRET	Validates X-Hub-Signature-256	Secret
WHATSAPP_VERIFY_TOKEN	Private string used during webhook challenge	Secret
WHATSAPP_PHONE_NUMBER_ID	Sender phone-number resource ID	Configuration
WHATSAPP_WABA_ID	WhatsApp Business Account ID	Configuration
WHATSAPP_GRAPH_VERSION	Pinned supported Graph API version	Configuration
WHATSAPP_SERVICE_TEMPLATE_NAME	Approved template for messages outside the service window	Configuration
WHATSAPP_SERVICE_TEMPLATE_LANGUAGE	Template locale, for example en_US	Configuration

```
const GRAPH_VERSION = process.env.WHATSAPP_GRAPH_VERSION ?? "vXX.X";
const GRAPH_BASE = `https://graph.facebook.com/${GRAPH_VERSION}`;

function required(name: string): string {
  const value = process.env[name];
  if (!value) throw new Error(`Missing required configuration: ${name}`);
  return value;
}
```

For multi-tenant applications, store one encrypted channel record per customer organization. Encrypt access tokens and app secrets at rest using an authenticated cipher such as AES-256-GCM. Keep the encryption root key in the secret store, not the database.

4. Recommended data model

Table	Essential fields	Important constraints
whatsapp_channels	tenant_id, waba_id, phone_number_id, encrypted token/secret, enabled, template config, health	Unique phone_number_id; tenant scoped
whatsapp_messages	tenant_id, customer phone, direction, body, provider_message_id, status timestamps	Unique provider_message_id and outbox_id
whatsapp_conversations	tenant_id, normalized phone, state, last inbound, opted_out_at, expires_at	Unique tenant_id + normalized phone
webhook_events	channel, external_id, received_at, processed_at	Unique channel + external_id
message_outbox	tenant_id, channel, payload, dedupe_key, attempts, state, next_attempt_at	Unique dedupe_key; atomic claim

Normalize phone numbers to digits in international format before using them as lookup keys. Preserve the original provider payload only when required for diagnostics, and apply retention limits.

5. Secure webhook implementation

5.1 Route ordering: preserve raw bytes

Meta's signature covers the exact request bytes. Mount the WhatsApp webhook with `express.raw()` **before** the global JSON parser.

```
import express from "express";

const app = express();

app.use(
  "/api/webhooks/whatsapp",
  express.raw({ type: "application/json", limit: "2mb" }),
  whatsappWebhookRouter,
);

app.use(express.json()); // mount after the raw webhook route
```

5.2 GET challenge verification

```
router.get("/", (req, res) => {
  const mode = req.query["hub.mode"];
  const token = req.query["hub.verify_token"];
  const challenge = req.query["hub.challenge"];

  if (!process.env.WHATSAPP_VERIFY_TOKEN) {
    return res.status(503).send("Webhook is not configured");
  }
  if (
    mode === "subscribe" &&
    token === process.env.WHATSAPP_VERIFY_TOKEN &&
    typeof challenge === "string"
  ) {
    return res.type("text/plain").status(200).send(challenge);
  }
  return res.sendStatus(403);
});
```

5.3 POST signature validation

```
import crypto from "node:crypto";

function verifyMetaSignature(
  rawBody: Buffer,
  header: string | undefined,
  appSecret: string,
): boolean {
  if (!header?.startsWith("sha256=")) return false;
  const supplied = Buffer.from(header.slice(7), "hex");
  const expected = crypto
    .createHmac("sha256", appSecret)
    .update(rawBody)
    .digest();
  return supplied.length === expected.length &&
    crypto.timingSafeEqual(supplied, expected);
}

router.post("/", async (req, res) => {
  const rawBody = Buffer.isBuffer(req.body)
    ? req.body
    : Buffer.from(req.body ?? "");
  const signature = req.get("X-Hub-Signature-256");

  if (!verifyMetaSignature(rawBody, signature, required("WHATSAPP_APP_SECRET"))) {
    return res.sendStatus(403);
  }

  let payload: unknown;
  try {
    payload = JSON.parse(rawBody.toString("utf8"));
  } catch {
    return res.sendStatus(400);
  }

  res.sendStatus(200); // acknowledge before slow work
  void processWebhookPayload(payload).catch(logger.error);
});
```

Multi-tenant signature validation: the phone-number ID is inside the signed body, but the body must not be trusted before signature verification. Safely try the small set of active app secrets, or use one platform app secret shared across channel records. Never select a secret from unverified payload data and assume it is valid.

6. Attach the app to the WABA

Execute this once during onboarding and again during a connection health check. A successful webhook configuration without this subscription may still receive no customer messages.

```
async function subscribeAppToWaba(wabaId: string, token: string) {
  const response = await fetch(
    `${GRAPH_BASE}/${encodeURIComponent(wabaId)}/subscribed_apps`,
    {
      method: "POST",
      headers: { Authorization: `Bearer ${token}` },
    },
  );
  const data = await response.json();
  if (!response.ok || data.success !== true) {
    throw new Error(`WABA subscription failed (${response.status})`);
  }
}
```

A robust onboarding test should also confirm that:

- the token is valid and has the expected app/system-user identity;
- the configured phone number belongs to the configured WABA;
- the phone uses the Cloud API platform and is connected;
- the application is subscribed to the WABA;
- the channel is not already assigned to another tenant.

7. Inbound message processing

Meta webhook payloads may contain multiple entries, changes, messages, and status receipts. Iterate all arrays; do not assume index zero is the only event.


```

type Inbound = {
  id: string;
  from: string;
  phoneNumberId: string;
  profileName?: string;
  type: string;
  text?: string;
  replyId?: string;
};

function extractInbound(payload: any): Inbound[] {
  const output: Inbound[] = [];
  for (const entry of payload?.entry ?? []) {
    for (const change of entry?.changes ?? []) {
      if (change?.field !== "messages") continue;
      const value = change.value ?? {};
      const phoneNumberId = value.metadata?.phone_number_id;
      const profileName = value.contacts?.[0]?.profile?.name;
      for (const msg of value.messages ?? []) {
        output.push({
          id: msg.id,
          from: msg.from,
          phoneNumberId,
          profileName,
          type: msg.type,
          text: msg.text?.body,
          replyId:
            msg.button?.payload ??
            msg.interactive?.button_reply?.id ??
            msg.interactive?.list_reply?.id,
        });
      }
    }
  }
  return output;
}

```

7.1 Idempotency

Meta retries webhooks, potentially for days. Atomically claim each message ID before processing it.

```

const claimed = await db.query(`
  INSERT INTO webhook_events (channel, external_id, received_at)
  VALUES ('meta_whatsapp', $1, now())
  ON CONFLICT (channel, external_id) DO NOTHING
  RETURNING external_id
`, [message.id]);

if (claimed.rowCount === 0) return; // duplicate delivery

```

Resolve the tenant exclusively from the verified `phone_number_id`. Unknown or disabled channels should be ignored or quarantined—never routed to a default tenant.

8. Outbound messages

8.1 Shared sender

```
type Channel = { phoneNumberId: string; accessToken: string };

async function sendMessage(channel: Channel, to: string, message: object) {
  const response = await fetch(
    `${GRAPH_BASE}/${channel.phoneNumberId}/messages`,
    {
      method: "POST",
      headers: {
        Authorization: `Bearer ${channel.accessToken}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        messaging_product: "whatsapp",
        recipient_type: "individual",
        to,
        ...message,
      }),
    },
  );
  const data = await response.json();
  if (!response.ok || !data.messages?.[0]?.id) {
    throw classifyProviderError(response.status, data);
  }
  return data.messages[0].id as string;
}
```

8.2 Text

```
await sendMessage(channel, customerPhone, {
  type: "text",
  text: { preview_url: false, body: "Thanks—how can we help?" },
});
```

8.3 Reply buttons

```
await sendMessage(channel, customerPhone, {
  type: "interactive",
  interactive: {
    type: "button",
    body: { text: "Would you like to book a test drive?" },
    action: {
      buttons: [
        { type: "reply", reply: { id: "book_yes", title: "Yes" } },
        { type: "reply", reply: { id: "book_later", title: "Not now" } },
      ],
    },
  },
});
```

Meta limits reply buttons and label lengths. Validate and truncate user-generated labels server-side.

8.4 List message

```
await sendMessage(channel, customerPhone, {
  type: "interactive",
  interactive: {
    type: "list",
    body: { text: "Choose a model to continue." },
    action: {
      button: "View models",
      sections: [{
        title: "Available models",
        rows: [
          { id: "model_1", title: "Model One", description: "SUV" },
          { id: "model_2", title: "Model Two", description: "Pickup" },
        ],
      }],
    },
  },
});
```

9. Customer-service window and templates

A customer message or call opens a rolling **24-hour customer-service window**. During that window, the business may send free-form service messages. Outside it, initiate contact only with an approved template in an eligible category and with the required consent.

```
const WINDOW_MS = 24 * 60 * 60 * 1000;

function hasOpenServiceWindow(lastInboundAt: Date | null): boolean {
  return !!lastInboundAt &&
    Date.now() - lastInboundAt.getTime() < WINDOW_MS;
}

if (hasOpenServiceWindow(conversation.lastInboundAt)) {
  await sendMessage(channel, to, {
    type: "text",
    text: { preview_url: false, body },
  });
} else {
  await sendMessage(channel, to, {
    type: "template",
    template: {
      name: channel.serviceTemplateName,
      language: { code: channel.serviceTemplateLanguage },
      components: [{
        type: "body",
        parameters: [{ type: "text", text: body }],
      }],
    },
  });
}
```

Do not bypass the window. A successful API call does not make an out-of-window free-form message compliant. Enforce this policy before enqueueing the send, and expose a clear “template required” state to staff.

Template category, content, consent, and pricing rules evolve. Reconfirm the current Meta policy before launch and whenever Meta announces a pricing or policy change.

10. Media and documents

For private business documents, upload the binary to Meta, receive a media ID, and then send that ID. Avoid public, long-lived document URLs.

```
async function uploadDocument(
  channel: Channel,
  bytes: Buffer,
  mimeType: string,
  filename: string,
): Promise<string> {
  const form = new FormData();
  form.append("messaging_product", "whatsapp");
  form.append("type", mimeType);
  form.append("file", new Blob([bytes], { type: mimeType }), filename);

  const response = await fetch(
    `${GRAPH_BASE}/${channel.phoneNumberId}/media`,
    { method: "POST", headers: {
      Authorization: `Bearer ${channel.accessToken}`,
    }, body: form },
  );
  const data = await response.json();
  if (!response.ok || !data.id) throw new Error("Media upload failed");
  return data.id;
}

const mediaId = await uploadDocument(channel, pdf, "application/pdf", "quote.pdf");
await sendMessage(channel, to, {
  type: "document",
  document: { id: mediaId, filename: "quote.pdf", caption: "Your quotation" },
});
```

Apply authorization before creating the document, avoid logging private storage paths, and retain a snapshot of the exact content represented by the outbound message.

11. Delivery and read receipts

The messages webhook also delivers status events such as sent, delivered, read, and failed. Persist the provider message ID returned by every send and correlate receipts to it.

- Deduplicate receipt events.
- Apply status transitions monotonically; do not regress *read* back to *delivered*.
- Treat provider-declared failures as terminal unless the error explicitly permits retry.
- Record timestamps separately for sent, delivered, read, and failed.
- Keep internal outbox correlation in `biz_opaque_callback_data` when supported.

12. Consent, STOP, and START

A customer-initiated inbound message supports service replies within the window; it is not blanket marketing consent. Maintain purpose-specific consent records for marketing campaigns.

```
const STOP = /^(stop|unsubscribe|opt[ -]?out)$/i;
const START = /^(start|unstop|resume)$/i;

if (STOP.test(text.trim())) {
  await markOptedOut(tenantId, phone);
  await sendComplianceAcknowledgement(phone,
    "You have been opted out. Reply START to resume.");
  return;
}
if (START.test(text.trim())) {
  await clearOptOut(tenantId, phone);
  await sendComplianceAcknowledgement(phone,
    "You are opted back in.");
  return;
}
```

Intercept compliance keywords before bots, menus, or AI. All outbound workers must check the opt-out state—not only the user interface.

13. Multi-tenant isolation

For software serving multiple businesses, tenant resolution is a security boundary.

- Map each verified phone-number ID to exactly one enabled tenant.
- Resolve inbound tenant context from that mapping—not from customer text, query parameters, or a global fallback.
- Resolve outbound credentials by tenant; never fall back to another tenant's sender.
- Stamp every conversation, message, event, lead, and outbox row with the tenant ID.
- Filter every read and write by tenant ID, including helper functions and background workers.
- Return no decrypted secrets through settings endpoints.

Fail closed. If no valid channel exists for a tenant or phone-number ID, stop. A “helpful” global fallback can send from the wrong business and expose customer data.

14. Optional conversational AI

Place AI above a controlled tool layer, not directly above the database or Graph API. The model should decide conversational intent; server-side tools should enforce authorization, tenancy, validation, and factual grounding.

Recommended tools include:

- search available inventory or product catalogue;
- retrieve the current customer's own enquiry/order status;
- create or update one deduplicated enquiry;
- generate a controlled appointment link;
- escalate to a human;
- save bounded, structured conversation memory.

Grounding requirements

- Force a fresh lookup for every availability, price, model, or status claim.
- Aggregate the complete authorized dataset before imposing display limits.
- Never allow the model to choose a tenant.
- Treat customer messages and model-generated tool arguments as untrusted input.
- Accept identity fields only when supported by customer-provided text.
- Fall back to a deterministic flow or human handoff if the model or tool fails.
- Keep the response concise and ask one question at a time.

AI inference and WhatsApp delivery may be billed separately. Track both provider costs.

15. Reliability and security controls

Durable outbox

Business transactions should enqueue messages in the same database transaction as the business event. A worker atomically claims rows, sends, records the provider ID, and retries eligible failures.

Error classification

Condition	Recommended disposition
HTTP 429	Retry with exponential backoff and jitter
HTTP 400/401/403	Usually terminal; flag configuration/authentication
HTTP 5xx	Retry with limits
Network timeout after request write	Outcome uncertain; reconcile using idempotency/correlation before retrying
Media upload failure before message send	Safe to retry upload

Security baseline

- Validate every POST signature using raw bytes and constant-time comparison.
- Use HTTPS only and keep the deployment publicly reachable for Meta.
- Use least-privilege tokens; rotate them and monitor expiry/revocation.
- Encrypt tenant credentials at rest with authenticated encryption.
- Rate-limit public routes without collapsing all reverse-proxy users into one IP bucket.
- Sanitize provider errors before showing them to users.
- Never log bearer tokens, app secrets, customer message bodies unnecessarily, or full provider payloads containing personal data.
- Apply retention policies to transcripts and webhook payloads.

16. Testing strategy

Point the Graph base URL to a local mock server. Use fake tokens and a dummy app secret—never production credentials.

Webhook tests

- ☐ GET challenge succeeds with the correct token and fails otherwise.
- ☐ Valid raw-body HMAC succeeds; modified body and bad signature fail.
- ☐ Malformed JSON returns 400.
- ☐ Unknown or disabled phone-number IDs fail closed.
- ☐ Duplicate message IDs execute business logic once.
- ☐ Text, button, list, media, and status payloads parse correctly.

Outbound tests

- ☐ Bearer authorization and endpoint path are correct.
- ☐ Text, button, list, template, media upload, and document payloads match Meta contracts.
- ☐ The 24-hour window allows free-form replies and requires templates after expiry.
- ☐ STOP blocks later sends; START resumes them.
- ☐ Retryable, terminal, and uncertain failures receive different dispositions.
- ☐ Delivery states cannot move backwards.

Isolation and AI tests

- ☐ Inbound events cannot cross tenants.
- ☐ Outbound messages never use another tenant's credentials.
- ☐ Inventory or catalogue totals use the complete authorized dataset.
- ☐ Model-only follow-ups trigger a fresh lookup.
- ☐ Prompt injection cannot reveal tools, credentials, or another customer's data.

17. Production deployment

1. Deploy the API to a stable public HTTPS URL.
2. Set production secrets through the platform's secret manager.
3. Apply the channel, transcript, event-ledger, conversation, and outbox database migrations.
4. Configure `https://YOUR_HOST/api/webhooks/whatsapp` in the Meta App Dashboard.
5. Complete the GET challenge using the production verify token.
6. Subscribe the `messages` field.
7. Attach the application to each WABA through `subscribed_apps`.
8. Send a customer-originated test message and confirm inbound persistence.
9. Reply inside the service window and confirm sent → delivered → read receipts.
10. Test the approved template outside the window using a consenting test recipient.
11. Verify STOP/START and cross-tenant isolation.

12. Enable monitoring for webhook failures, token health, queue age, retries, and undelivered messages.

Webhook availability: private or password-gated deployments are not directly reachable by Meta unless the hosting platform provides a supported external-access mechanism. Verify external reachability before declaring the integration complete.

18. Troubleshooting

Symptom	Likely cause	Resolution
GET verification fails	Token mismatch, wrong path, non-public deployment	Compare exact verify token; curl the public HTTPS URL
POST always returns 403	JSON parser changed bytes or wrong app secret	Mount raw parser first; verify HMAC over exact bytes
Webhook configured but no inbound events	App not subscribed to WABA	Call POST <code>/WABA_ID/subscribed_apps</code>
Messages accepted but customer receives nothing	Wrong phone ID, template issue, recipient restriction, or later failed receipt	Inspect provider response and status webhook
Free-form message rejected	24-hour window expired	Use an approved, eligible template
Customers receive duplicates	Webhook retries or non-idempotent outbox	Unique event IDs and dedupe keys; atomic claims
Wrong business replies	Global credential fallback	Resolve channel strictly by verified phone ID/tenant
AI gives stale product information	Skipped lookup or truncated data before aggregation	Force a fresh tool call and aggregate before limiting
Token suddenly stops working	Revoked/expired token or missing permissions	Run token debug/health check and rotate safely

19. Production readiness checklists

Meta configuration

- ☐ Business, WABA, and phone number are approved and connected.
- ☐ Production system-user token uses minimum permissions.
- ☐ Webhook callback and verify token are configured.
- ☐ messages is subscribed and the app is attached to the WABA.
- ☐ Required templates are approved in every needed language.

Application

- ☐ Raw-body signature validation is mandatory.
- ☐ Inbound events and status receipts are idempotent.
- ☐ Outbound sends use a durable outbox and classified retries.
- ☐ Service-window and opt-out rules are enforced server-side.
- ☐ Transcripts and delivery states are persisted.
- ☐ Tenant/channel resolution fails closed.
- ☐ Secrets are encrypted, redacted, and never returned to clients.

Operations

- ☐ Token and channel health are monitored.
- ☐ Webhook latency/error rate and outbox queue age are monitored.
- ☐ Support staff can identify failed messages without seeing secrets.
- ☐ Consent, retention, privacy, and deletion policies are documented.
- ☐ Meta pricing and policy changes have an assigned owner.

20. Official Meta references

- [WhatsApp Cloud API — Get Started](#)
- [Webhooks overview](#)
- [Messages webhook reference](#)
- [Service messages and customer-service windows](#)
- [Template fundamentals](#)
- [Template review](#)
- [WhatsApp Business Platform pricing](#)

Document status: Reference implementation guide, September 2026. Meta APIs, supported Graph versions, template rules, pricing, and permissions change over time. Pin a supported Graph version and validate current official documentation before each production launch. This guide is technical guidance, not legal advice.